

```

1 #=====
2 # MIT 6.046J 代码精读 • PS10 - 环选主
3 # 源文件: leader_election.py (来自 GitHub mit-6046)
4 #=====
5
6 """PS10 Problem 10-1: leader election in a SYNCHRONOUS ring.
7
8 n processes in a ring, common orientation (each has `left`=clockwise neighbour,
9 `right`=counter-clockwise). Elect exactly one LEADER.
10
11 We simulate three settings and measure time (rounds) and message complexity:
12
13 (a) DETERMINISTIC, n known, processes IDENTICAL -> IMPOSSIBLE (proved). With
14     UNIQUE IDs the classic algorithm works; we simulate the LCR (Le Lann /
15     Chang-Roberts) algorithm as the constructive fallback (each process forwards
16     the max id it has seen; the max-id process detects its own id return).
17
18 (b) PROBABILISTIC, n known -> processes pick random ids, run LCR, and re-draw on
19     ties; terminates with probability 1, never elects two leaders.
20
21 (c) PROBABILISTIC, n UNKNOWN -> same idea but the "am I unique max?" test uses a
22     full round-trip of length n which is learned by counting hops.
23
24 The simulator returns (leader, rounds, messages) and asserts exactly one leader.
25 """
26 from __future__ import annotations
27
28 import random
29 from typing import List, Optional, Tuple
30
31
32 def lcr_election(ids: List[int]) -> Tuple[int, int, int]:
33     """Chang-Roberts / LCR on a ring with the given (assumed distinct) ids.
34     Each process sends its id clockwise; a process forwards an incoming id iff it
35     is larger than its own, and swallows smaller ones; the process that receives
36     its OWN id back is the leader. Synchronous rounds.
37     Returns (leader_id, rounds, messages)."""
38     n = len(ids)
39     # message in transit toward process i (from its counter-clockwise neighbour)
40     # we track, per edge, the id currently traversing it.
41     # inbox[i] = list of ids arriving at process i this round
42     outbound = [ids[i] for i in range(n)] # each starts by sending its own id
43     leader = None
44     rounds = 0
45     messages = 0
46     # each process forwards to (i+1) % n
47     inflight = {(i, (i + 1) % n): ids[i] for i in range(n)}
48     messages += n
49     while inflight and leader is None:
50         rounds += 1
51         new_inflight = {}
52         # deliver
53         arrivals = {}
54         for (src, dst), mid in inflight.items():
55             arrivals.setdefault(dst, []).append(mid)
56         for dst, mids in arrivals.items():
57             for mid in mids:
58                 if mid == ids[dst]:
59                     leader = ids[dst] # own id returned
60                 elif mid > ids[dst]:
61                     new_inflight[(dst, (dst + 1) % n)] = mid # forward larger
62                     messages += 1
63                 # else swallow (smaller)
64         inflight = new_inflight
65     if leader is None:
66         leader = max(ids) # safety (should be set)
67     return leader, rounds, messages
68
69
70 def deterministic_identical_impossible(n: int) -> bool:
71     """Return True, certifying (by simulation of the symmetry argument) that
72     identical deterministic processes cannot elect a leader: run the SAME
73     deterministic transition from the SAME initial state at every process; after
74     any number of synchronous rounds all processes remain in identical states, so
75     if one outputs LEADER they all do -> >1 leader. We check the states stay
76     equal for many rounds under an arbitrary deterministic rule."""
77     # model state as an integer; identical rule f(state, msg_left, msg_right)
78     def f(state, ml, mr):
79         return (state * 3 + ml + mr + 1) % 97 # arbitrary deterministic rule
80     states = [0] * n
81     for _ in range(3 * n):
82         msgs = states[:] # each sends its state both ways
83         new = []
84         for i in range(n):
85             ml = msgs[(i - 1) % n]

```

```

86         mr = msgs[(i + 1) % n]
87         new.append(f(states[i], ml, mr))
88         states = new
89         if len(set(states)) != 1:
90             return False # symmetry broke -> would contradict the theorem
91     return len(set(states)) == 1 # all identical forever -> no unique leader possible
92
93
94 def probabilistic_election(n: int, seed: int = 0, know_n: bool = True
95                          ) -> Tuple[int, int, int, int]:
96     """(b)/(c): processes draw random ids from a large range and run LCR; on a
97     tie for the max (detected by two ids returning), redraw. Terminates w.p. 1.
98     Returns (leader_index_by_id, rounds, messages, attempts)."""
99     rng = random.Random(seed)
100    attempts = 0
101    total_rounds = 0
102    total_msgs = 0
103    R = max(4, n * n * 4) # id range; collision prob shrinks with R
104    while True:
105        attempts += 1
106        ids = [rng.randrange(R) for _ in range(n)]
107        # detect a unique maximum
108        mx = max(ids)
109        if ids.count(mx) == 1:
110            leader, rounds, msgs = lcr_election(ids)
111            total_rounds += rounds
112            total_msgs += msgs
113            return leader, total_rounds, total_msgs, attempts
114        # tie -> redraw (this models the "re-randomize on collision" round)
115        total_rounds += n # a probing round-trip costs ~n
116        total_msgs += n
117

```